

Tecnología Digital VI

LSTM, Seq2Seq, Attention

Licenciatura en Tecnología Digital - UTDT

Más que texto

Más allá de sus problemas, la introducción de las RNNs sirve como un formalismo que busca aprender sobre secuencias de datos en general, no solo texto.

Si bien cada nicho de aplicación tiene sus particularidades (y luego sus técnicas específicas), el procesamiento de secuencias puede tener múltiples casos de uso:

- Procesamiento de audio.
- Series de tiempo.
- Secuencias de aminoácidos (proteínas).

SPEECH RECOGNITION WITH DEEP RECURRENT NEURAL NETWORKS

Alex Graves, Abdel-rahman Mohamed and Geoffrey Hinton

Department of Computer Science, University of Toronto

ABSTRACT

Recurrent neural networks (RNNs) are a powerful model for sequential data. End-to-end training methods such as Connectionist Temporal Classification make it possible to train RNNs for sequence labelling problems where the input-output alignment is unknown. The combination of these methods with the Long Short-term Memory RNN architecture has proved particularly fruitful, delivering state-of-the-art results in cursive handwriting recognition. However RNN performance in speech recognition has so far been disappointing, with better results returned by deep feedforward networks. This paper investigates *deep recurrent neural networks*, which combine the multiple levels of representation that have proved so effective in deep networks with the flexible use of long range context that empowers RNNs. When trained end-to-end with suitable regularisation, we find that deep Long Short-term Memory RNNs achieve a test set error of 17.7% on the TIMIT phoneme recognition benchmark, which to our knowledge is the best recorded score.

Index Terms— recurrent neural networks, deep neural networks, speech recognition

1. INTRODUCTION

RNNs are inherently deep in time, since their hidden state is a function of all previous hidden states. The question that inspired this paper was whether RNNs could also benefit from depth in space; that is from stacking multiple recurrent hidden layers on top of each other, just as feedforward layers are stacked in conventional deep networks. To answer this question we introduce *deep Long Short-term Memory* RNNs and assess their potential for speech recognition. We also present an enhancement to a recently introduced end-to-end learning method that jointly trains two separate RNNs as acoustic and linguistic models [10]. Sections 2 and 3 describe the network architectures and training methods, Section 4 provides experimental results and concluding remarks are given in Section 5.

2. RECURRENT NEURAL NETWORKS

Given an input sequence $x = (x_1, \dots, x_T)$, a standard recurrent neural network (RNN) computes the hidden vector sequence $h = (h_1, \dots, h_T)$ and output vector sequence $y = (y_1, \dots, y_T)$ by iterating the following equations from $t = 1$ to T :

$$h_t = \mathcal{H}(W_{xh}x_t + W_{hh}h_{t-1} + b_h) \quad (1)$$

$$y_t = W_{hy}h_t + b_y \quad (2)$$

WAVENET: A GENERATIVE MODEL FOR RAW AUDIO

Aäron van den Oord	Sander Dieleman	Heiga Zen [†]
Karen Simonyan	Oriol Vinyals	Alex Graves
Nal Kalchbrenner	Andrew Senior	Koray Kavukcuoglu

{avdnoord, sedielem, heigazem, simonyan, vinyals, graves, nalk, andrewsenior, korayk}@google.com
Google DeepMind, London, UK

[†] Google, London, UK

ABSTRACT

This paper introduces WaveNet, a deep neural network for generating raw audio waveforms. The model is fully probabilistic and autoregressive, with the predictive distribution for each audio sample conditioned on all previous ones; nonetheless we show that it can be efficiently trained on data with tens of thousands of samples per second of audio. When applied to text-to-speech, it yields state-of-the-art performance, with human listeners rating it as significantly more natural sounding than the best parametric and concatenative systems for both English and Mandarin. A single WaveNet can capture the characteristics of many different speakers with equal fidelity, and can switch between them by conditioning on the speaker identity. When trained to model music, we find that it generates novel and often highly realistic musical fragments. We also show that it can be employed as a discriminative model, returning promising results for phoneme recognition.

Deep Speech: Scaling up end-to-end speech recognition

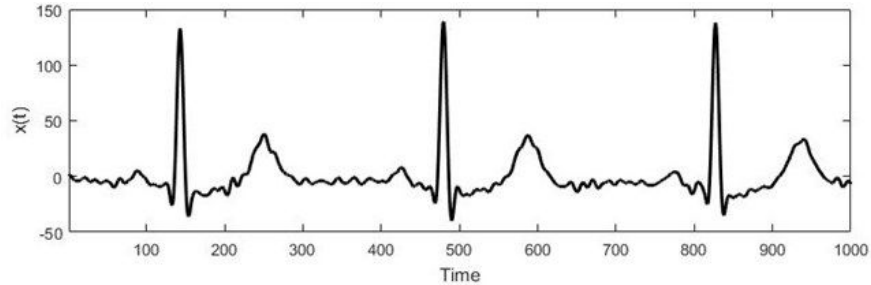
Awni Hannun*, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, Andrew Y. Ng

Baidu Research – Silicon Valley AI Lab

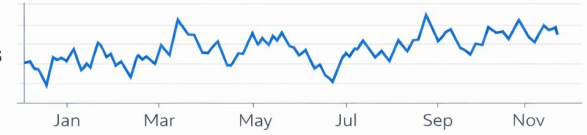
Abstract

We present a state-of-the-art speech recognition system developed using end-to-end deep learning. Our architecture is significantly simpler than traditional speech systems, which rely on laboriously engineered processing pipelines; these traditional systems also tend to perform poorly when used in noisy environments. In contrast, our system does not need hand-designed components to model background noise, reverberation, or speaker variation, but instead directly learns a function that is robust to such effects. We do not need a phoneme dictionary, nor even the concept of a “phoneme.” Key to our approach is a well-optimized RNN training system that uses multiple GPUs, as well as a set of novel data synthesis techniques that allow us to efficiently obtain a large amount of varied data for training. Our system, called Deep Speech, outperforms previously published results on the widely studied Switchboard Hub5*00, achieving 16.0% error on the full test set. Deep Speech also handles challenging noisy environments better than widely used, state-of-the-art commercial speech systems.

Series de tiempo



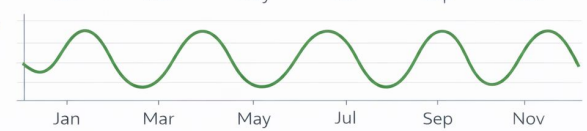
Original
Time Series



Trend



Seasonality



Residuals



Series de tiempo

dmlc
XGBoost

PROPHET


Forecasting at Scale

Sean J. Taylor and Benjamin Letham

January 13, 2017

Abstract

There are a variety of challenges that come with producing a large number of forecasts across a variety time series. Our approach to forecasting *at scale* is a combination of configurable models and thorough analyst-in-the-loop performance analysis. We present a forecasting approach based on a decomposable model with interpretable parameters that can be intuitively adjusted by the analyst. We describe performance analysis that we use compare and evaluate forecasting procedures, as well as automatically flag forecasts for manual review and adjustment. Tools that help analysts to use their expertise most effectively enable reliable forecasting of a large variety of business time series.

A decoder-only foundation model for time-series forecasting

Abhimanyu Das

Weihao Kong

Rajat Sen

Yichen Zhou

Abstract

Motivated by recent advances in large language models for Natural Language Processing (NLP), we design a time-series foundation model for forecasting whose out-of-the-box zero-shot performance on a variety of public datasets comes close to the accuracy of state-of-the-art supervised forecasting models for each individual dataset. Our model is based on pretraining a patched-decoder style attention model on a large time-series corpus comprising both real-world and synthetic datasets. Experiments on a diverse set of previously unseen forecasting datasets suggests that the model can yield accurate zero-shot forecasts across different domains, forecasting horizons and temporal granularities.

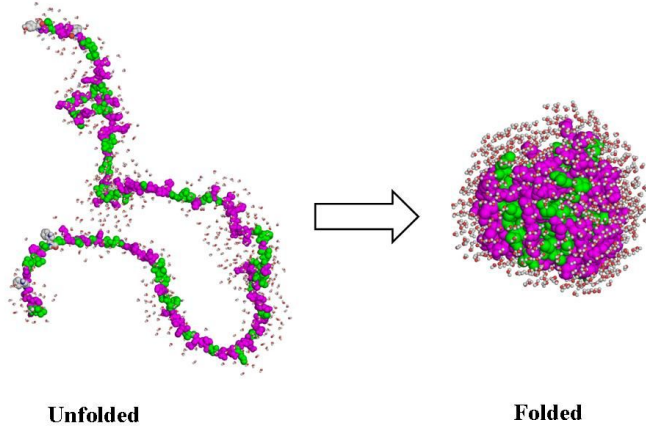
DeepAR: Probabilistic Forecasting with Autoregressive Recurrent Networks

David Salinas, Valentin Flunkert, Jan Gasthaus
Amazon Research
Germany
<dsalina,flunkert,gasthaus@amazon.com>

Abstract

Probabilistic forecasting, i.e. estimating the probability distribution of a time series' future given its past, is a key enabler for optimizing business processes. In retail businesses, for example, forecasting demand is crucial for having the right inventory available at the right time at the right place. In this paper we propose DeepAR, a methodology for producing accurate probabilistic forecasts, based on training an auto-regressive recurrent network model on a large number of related time series. We demonstrate how by applying deep learning techniques to forecasting, one can overcome many of the challenges faced by widely-used classical approaches to the problem. We show through extensive empirical evaluation on several real-world forecasting data sets accuracy improvements of around 15% compared to state-of-the-art methods.

Protein Folding



Improved protein structure prediction using potentials from deep learning

[Andrew W. Senior](#) ✉, [Richard Evans](#), [John Jumper](#), [James Kirkpatrick](#), [Laurent Sifre](#), [Tim Green](#), [Chongli Qin](#), [Augustin Židek](#), [Alexander W. R. Nelson](#), [Alex Bridgland](#), [Hugo Penedones](#), [Stig Petersen](#), [Karen Simonyan](#), [Steve Crossan](#), [Pushmeet Kohli](#), [David T. Jones](#), [David Silver](#), [Koray Kavukcuoglu](#) & [Demis Hassabis](#)

[Nature](#) **577**, 706–710 (2020) | [Cite this article](#)

202k Accesses | 3285 Citations | 783 Altmetric | [Metrics](#)

Article | Published: 27 April 2022

Machine learning-aided engineering of hydrolases for PET depolymerization

[Hongyuan Lu](#), [Daniel J. Diaz](#), [Natalie J. Czarnecki](#), [Congzhi Zhu](#), [Wantae Kim](#), [Raghav Shroff](#), [Daniel J. Acosta](#), [Bradley R. Alexander](#), [Hannah O. Cole](#), [Yan Zhang](#), [Nathaniel A. Lynd](#), [Andrew D. Ellington](#) & [Hal S. Alper](#) ✉

[Nature](#) **604**, 662–667 (2022) | [Cite this article](#)

Vimos que el procesamiento de secuencias tiene un montón de aplicaciones diversas, pero volvamos a nuestro formalismo para resolver estos problemas.

Las RNNs en su definición tienen la capacidad teórica de aprender sobre valores previos específicos que se vieron en la secuencia pero en la práctica ese aprendizaje se encuentra limitado a datos no muy alejados en el tiempo.

El siguiente paso en la búsqueda de mejores modelos para estos problemas fueron las redes LSTM (Long Short-Term Memory), las cuales son un tipo de RNNs con algunos componentes adicionales.

Al igual que sucedió con las CNNs, las LSTM fueron introducidas muchos años antes de que explotaran las aplicaciones prácticas con ellas.

La arquitectura se introdujo originalmente en 1997 como una potencial solución al problema de “decaying error back flow”.

“LSTM can be learn to bridge minimal time lags in excess of 1000 discrete time steps by enforcing *constant error flow*”.

LONG SHORT-TERM MEMORY

NEURAL COMPUTATION 9(8):1735–1780, 1997

Sepp Hochreiter
Fakultät für Informatik
Technische Universität München
80290 München, Germany
hochreit@informatik.tu-muenchen.de
<http://www7.informatik.tu-muenchen.de/~hochreit>

Jürgen Schmidhuber
IDSIA
Corso Elvezia 36
6900 Lugano, Switzerland
juergen@idsia.ch
<http://www.idsia.ch/~juergen>

Abstract

Learning to store information over extended time intervals via recurrent backpropagation takes a very long time, mostly due to insufficient, decaying error back flow. We briefly review Hochreiter's 1991 analysis of this problem, then address it by introducing a novel, efficient, gradient-based method called “Long Short-Term Memory” (LSTM). Truncating the gradient where this does not do harm, LSTM can learn to bridge minimal time lags in excess of 1000 discrete time steps by enforcing *constant error flow* through “constant error carousels” within special units. Multiplicative gate units learn to open and close access to the constant error flow. LSTM is local in space and time; its computational complexity per time step and weight is $O(1)$. Our experiments with artificial data involve local, distributed, real-valued, and noisy pattern representations. In comparisons with RTRL, BPTT, Recurrent Cascade-Correlation, Elman nets, and Neural Sequence Chunking, LSTM leads to many more successful runs, and learns much faster. LSTM also solves complex, artificial long time lag tasks that have never been solved by previous recurrent network algorithms.

LSTM

En las LSTM ahora tenemos dos vectores de información que pasan de un instante t al siguiente. Por un lado tenemos el mismo *hidden state* H_t que teníamos en las primeras RNNs. Por el otro, tenemos el *cell state* C_t .

Las LSTM tiene componentes particulares (*gates*) que sirven para *leer*, *borrar* y *escribir* estos estados de manera dinámica según el contexto de lo que se está procesando en la red.

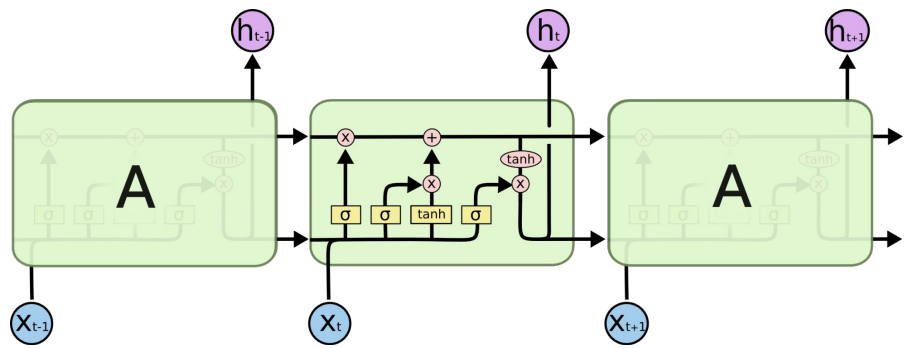
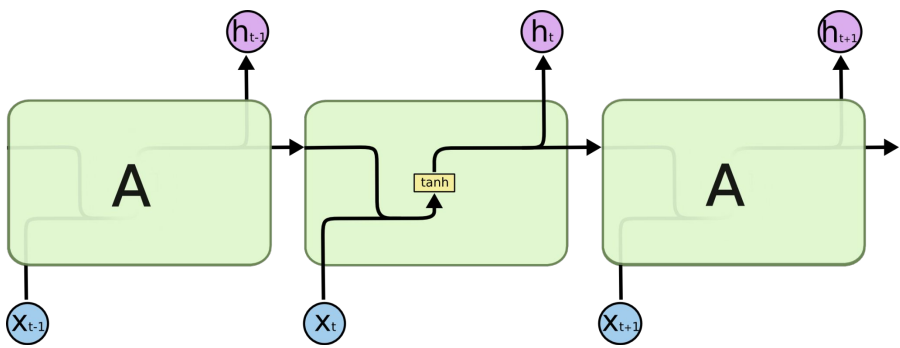
La idea es que el vector h sirva como una memoria a corto plazo del contexto exacto actual de lo que se viene procesando, mientras que el vector c debería servir como una memoria a más largo plazo dado que, por como están definidas sus cuentas, tiene la capacidad de pasar la información de hace varias palabras atrás sin que esta se *desvanezca*.

Las *gates* o *compuertas* de las LSTM son también vectores del mismo tamaño que c y h que deciden cuánta información fluye entre ellos, hacia el output y hacia las próximas iteraciones.

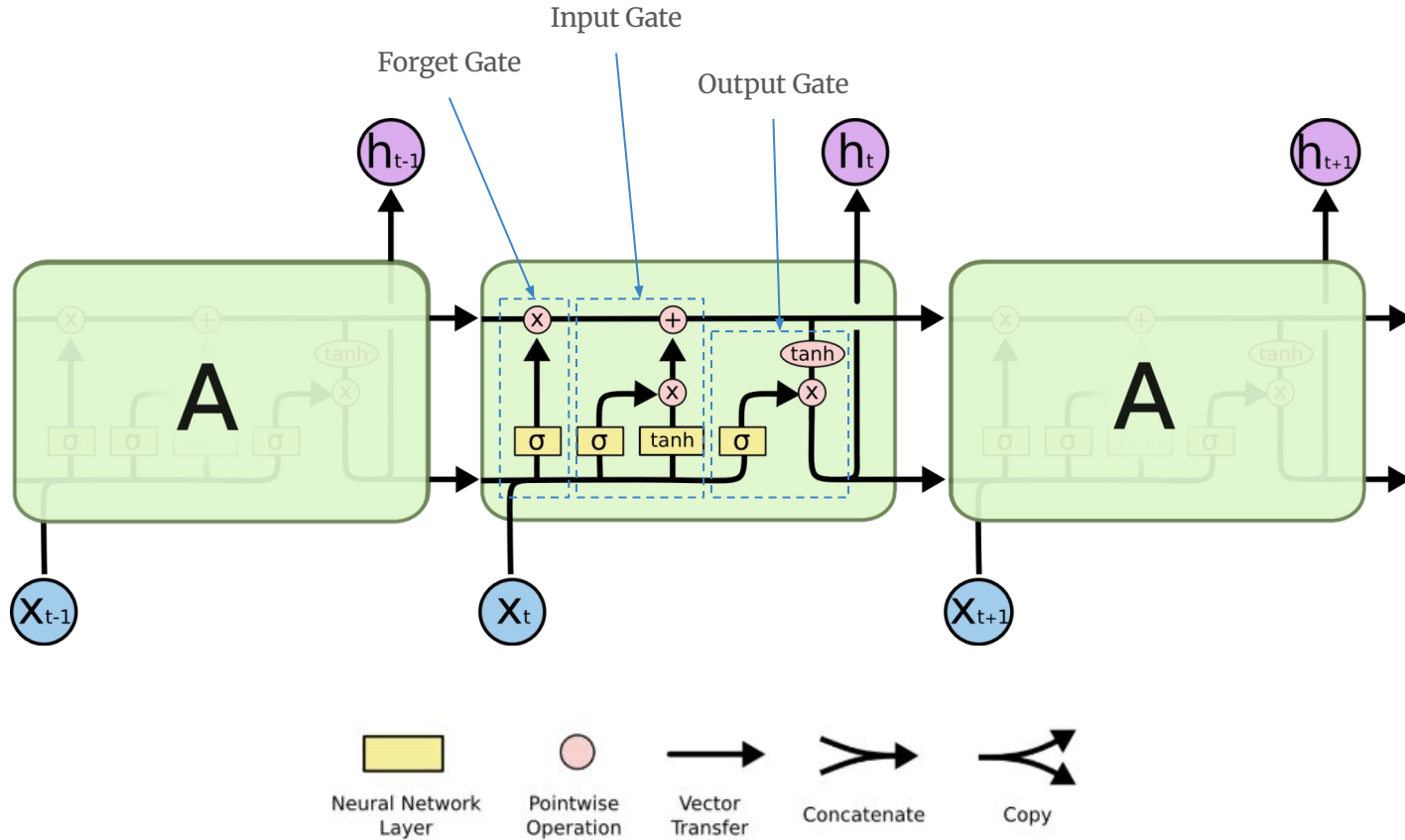
Si bien hay diferentes tipos de LSTM las *gates* más comunes son:

- Forget Gate: Decide qué borrar del vector c .
- Input Gate: Decide cuánto de lo *actual* se escribe en el vector c .
- Output Gate: Decide cuánto del vector c se *lee* para afectar al vector h (y en consecuencia, a la salida de la red).

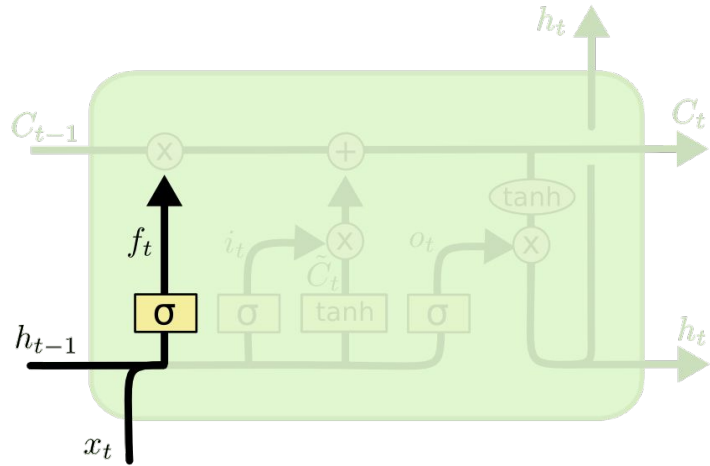
RNN vs LSTM



LSTM

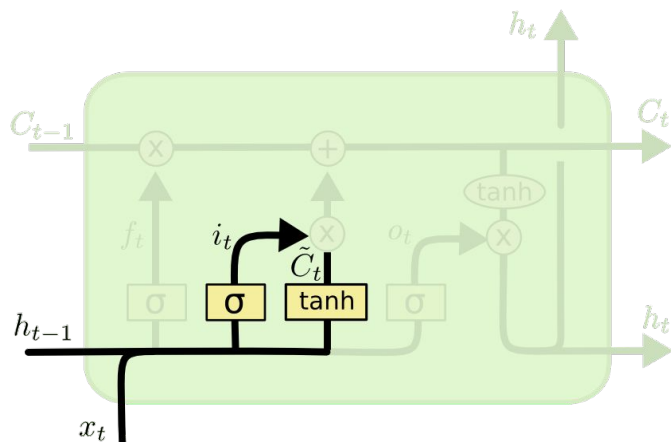


Forget Gate



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

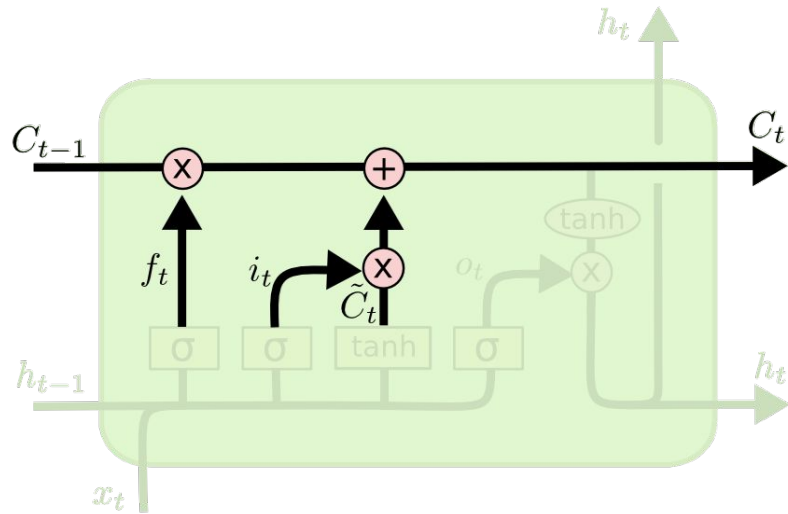
Input Gate



$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Forget+Input

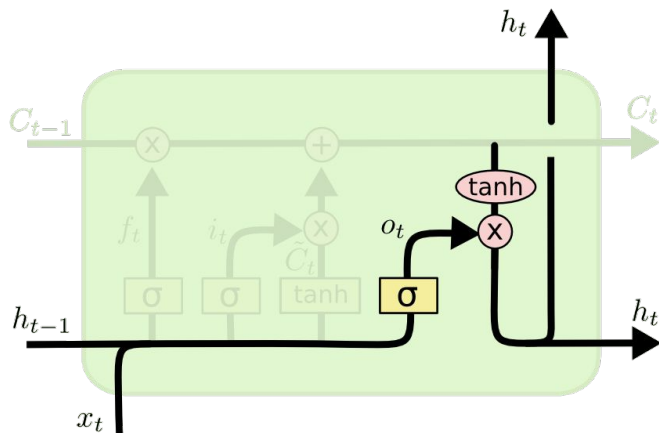


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Notación opcional

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Output Gate



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

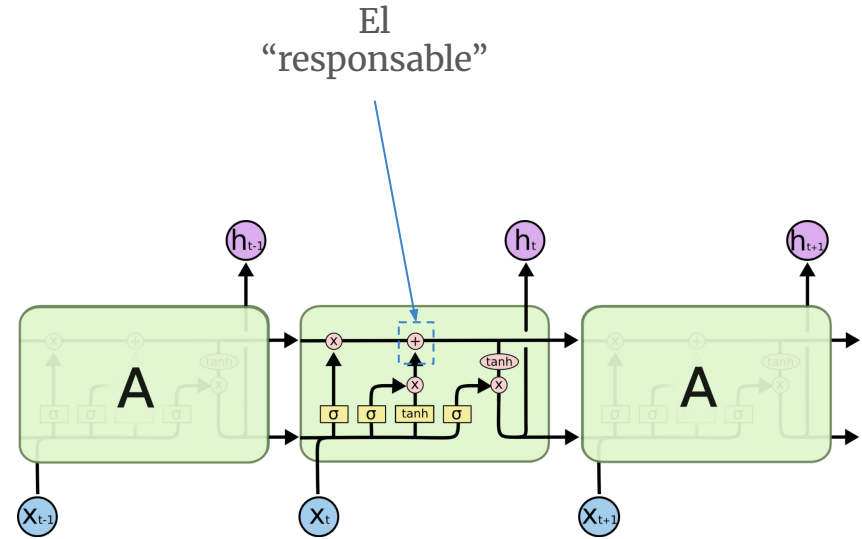
$$h_t = o_t * \tanh (C_t)$$

Resolviendo *vanishing gradients*

El “+” en el cómputo del vector C hace que la información pueda fluir por la red varios pasos hacia adelante sin desvanecerse.

Yendo al caso extremo (cómo hacíamos en las capas residuales), si seteamos la forget gate y la input gate en 1 y 0 adecuadamente, el vector C fluye completo.

Esto en la práctica hizo que fue posible pasar de 6 o 7 pasos de tiempo al orden de las centenas.



Semántica de las compuertas

“Input”, “Forget”, “Output”, son todos nombres que utilizamos para caracterizar las fórmulas pero para el formalismo no son más que otras matrices de pesos que tiene que aprender.

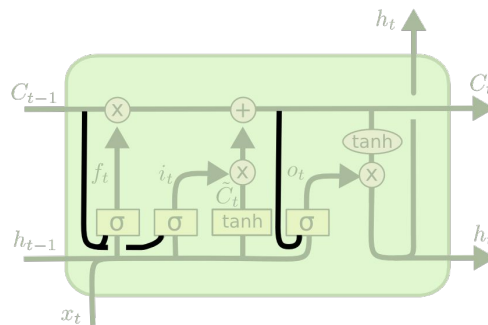
¿Cómo es que la estamos diciendo a la red “esto usalo para recordar algo muy viejo”?

De alguna manera, podemos decir que no lo estamos haciendo, pero si hay algo interesante de información a traer de una palabra que está 100 posiciones la *única* manera va a ser a través del vector c y, por como son las cuentas, la única manera de que alguien te diga que tenés que recordar esa palabra es mediante la compuerta *forget*.

Variantes de LSTM

Las nociones de la LSTM presentada se pueden combinar de diferentes maneras para crear otras posibles LSTM (de hecho en el paper original no existía la *forget gate*).

Una alternativa posible es dejar que las compuertas también vean el *cell state* para computarse.



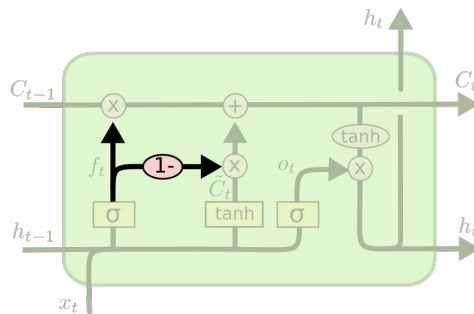
$$f_t = \sigma(W_f \cdot [C_{t-1}, h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i \cdot [C_{t-1}, h_{t-1}, x_t] + b_i)$$

$$o_t = \sigma(W_o \cdot [C_t, h_{t-1}, x_t] + b_o)$$

Variantes de LSTM

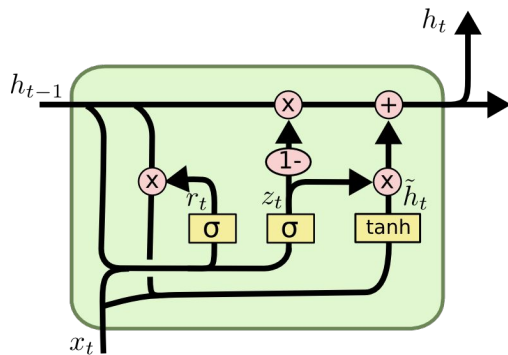
Otra alternativa posible es que en las *forget* e *input gates* estén muy acopladas para que la *forget gate* solo olvide las partes del vector c que efectivamente van a ser sobrescritas ahora por la *input gate*.



$$C_t = f_t * C_{t-1} + (1 - f_t) * \tilde{C}_t$$

Otra alternativa que tuvo mucho reconocimiento en su momento fueron las GRUs (Gated Recurrent Units).

Es una versión simplificada de las LSTM en donde los vectores c y h se combinan en uno solo y las compuertas *forget* e *input* también.



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Hoy en imágenes los problemas estrellas son la generación, la detección de objetos, el tracking, la segmentación, pero originalmente el problema que dió el salto cualitativo importante fue la clasificación y el dataset ImageNet.

El análogo en texto es el problema de Machine Translation. Hubo muchos problemas antes y hay muchos problemas después pero este es posiblemente “*el ImageNet*” del mundo del Procesamiento del Lenguaje Natural (NLP).

En el problema de traducción tenemos que convertir secuencias en secuencias. La entrada va a ser toda una secuencia de texto, y la salida también.

Hasta el 2014, los traductores automáticos funcionaban con modelos conocidos dentro del espectro “Statistical Machine Translation”, formulas de probabilidad condicional entrenadas sobre muchas traducciones reales.

En el 2014, gracias al paper de Sutskever, el paradigma mutó al Neural Machine Translation.

Sequence to Sequence Learning with Neural Networks

Ilya Sutskever
Google
ilyasu@google.com

Oriol Vinyals
Google
vinyals@google.com

Quoc V. Le
Google
qvl@google.com

Abstract

Deep Neural Networks (DNNs) are powerful models that have achieved excellent performance on difficult learning tasks. Although DNNs work well whenever large labeled training sets are available, they cannot be used to map sequences to sequences. In this paper, we present a general end-to-end approach to sequence learning that makes minimal assumptions on the sequence structure. Our method uses a multilayered Long Short-Term Memory (LSTM) to map the input sequence to a vector of a fixed dimensionality, and then another deep LSTM to decode the target sequence from the vector. Our main result is that on an English to French translation task from the WMT'14 dataset, the translations produced by the LSTM achieve a BLEU score of 34.8 on the entire test set, where the LSTM's BLEU score was penalized on out-of-vocabulary words. Additionally, the LSTM did not have difficulty on long sentences. For comparison, a phrase-based SMT system achieves a BLEU score of 33.3 on the same dataset. When we used the LSTM to rerank the 1000 hypotheses produced by the aforementioned SMT system, its BLEU score increases to 36.5, which is close to the previous best result on this task. The LSTM also learned sensible phrase and sentence representations that are sensitive to word order and are relatively invariant to the active and the passive voice. Finally, we found that reversing the order of the words in all source sentences (but not target sentences) improved the LSTM's performance markedly, because doing so introduced many short term dependencies between the source and the target sentence which made the optimization problem easier.

Si las LSTM tenían tanto tiempo dando vueltas, ¿Por qué recién en el 2014 se combinaron 2 LSTM para tener tan buenos resultados?

La parte más importante de la respuesta:
Word2Vec.

Word2Vec mostró que las capas de embeddings tenían mucho más sentido y semántica, que no hacía falta trabajar con OHE vectors de tamaño tan grande como el vocabulario.

La combinación de la semántica de las *gates* en la LSTM con estos espacios latentes (y la noción de espacio latente en sí) fueron los grandes cambios.

2 The model

The Recurrent Neural Network (RNN) [31, 28] is a natural generalization of feedforward neural networks to sequences. Given a sequence of inputs $(x_1, \dots, x_T)$, a standard RNN computes a sequence of outputs $(y_1, \dots, y_T)$ by iterating the following equation:

$$\begin{aligned}h_t &= \text{sigm}(W^{\text{hx}}x_t + W^{\text{hh}}h_{t-1}) \\y_t &= W^{\text{yh}}h_t\end{aligned}$$

The RNN can easily map sequences to sequences whenever the alignment between the inputs the outputs is known ahead of time. However, it is not clear how to apply an RNN to problems whose input and the output sequences have different lengths with complicated and non-monotonic relationships.

The simplest strategy for general sequence learning is to map the input sequence to a fixed-sized vector using one RNN, and then to map the vector to the target sequence with another RNN (this approach has also been taken by Cho et al. [5]). While it could work in principle since the RNN is provided with all the relevant information, it would be difficult to train the RNNs due to the resulting long term dependencies (figure 1) [14, 4, 16, 15]. However, the Long Short-Term Memory (LSTM) [16] is known to learn problems with long range temporal dependencies, so an LSTM may succeed in this setting.

The goal of the LSTM is to estimate the conditional probability $p(y_1, \dots, y_{T'} | x_1, \dots, x_T)$ where $(x_1, \dots, x_T)$ is an input sequence and $y_1, \dots, y_{T'}$ is its corresponding output sequence whose length T' may differ from T . The LSTM computes this conditional probability by first obtaining the fixed-dimensional representation v of the input sequence $(x_1, \dots, x_T)$ given by the last hidden state of the LSTM, and then computing the probability of $y_1, \dots, y_{T'}$ with a standard LSTM-LM formulation whose initial hidden state is set to the representation v of $x_1, \dots, x_T$:

$$p(y_1, \dots, y_{T'} | x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t | v, y_1, \dots, y_{t-1}) \quad (1)$$

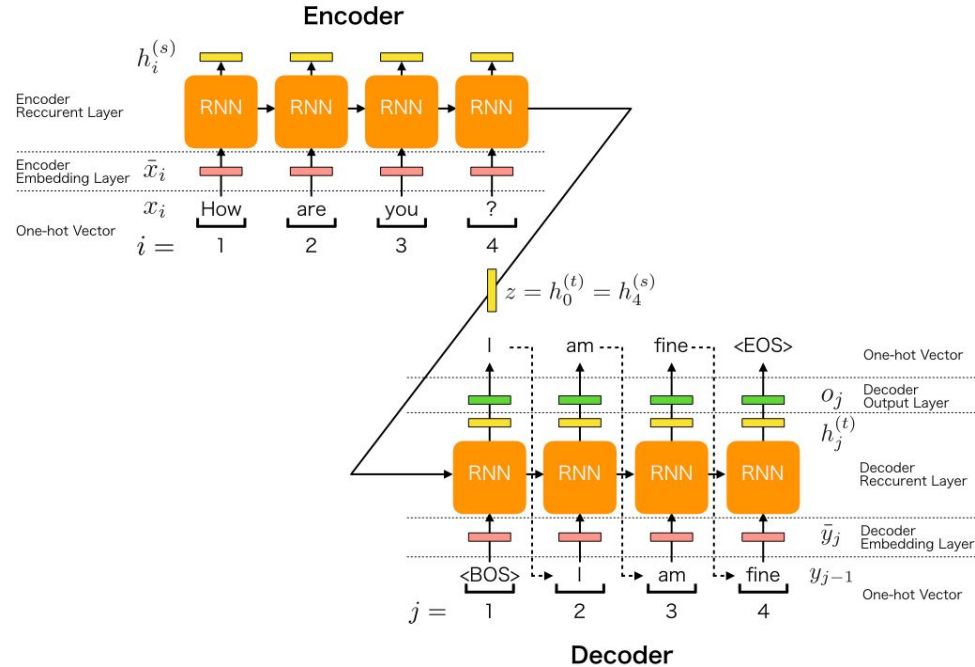
In this equation, each $p(y_t | v, y_1, \dots, y_{t-1})$ distribution is represented with a softmax over all the words in the vocabulary. We use the LSTM formulation from Graves [10]. Note that we require that each sentence ends with a special end-of-sentence symbol “<EOS>”, which enables the model to define a distribution over sequences of all possible lengths. The overall scheme is outlined in figure 1, where the shown LSTM computes the representation of “A”, “B”, “C”, “<EOS>” and then uses this representation to compute the probability of “W”, “X”, “Y”, “Z”, “<EOS>”.

Our actual models differ from the above description in three important ways. First, we used two different LSTMs: one for the input sequence and another for the output sequence, because doing so increases the number model parameters at negligible computational cost and makes it natural to train the LSTM on multiple language pairs simultaneously [18]. Second, we found that deep LSTMs significantly outperformed shallow LSTMs, so we chose an LSTM with four layers. Third, we found it extremely valuable to reverse the order of the words of the input sentence. So for example, instead of mapping the sentence a, b, c to the sentence α, β, γ , the LSTM is asked to map c, b, a to α, β, γ , where α, β, γ is the translation of a, b, c . This way, a is in close proximity to α , b is fairly close to β , and so on, a fact that makes it easy for SGD to “establish communication” between the input and the output. We found this simple data transformation to greatly improve the performance of the LSTM.

Seq2Seq

El modelo consistía en dos redes LSTM, un *encoder* y un *decoder*. La frase original se pasa por el *encoder* palabra a palabra generando un estado latente que luego es utilizado en el decoder para ir generando palabra a palabra la salida.

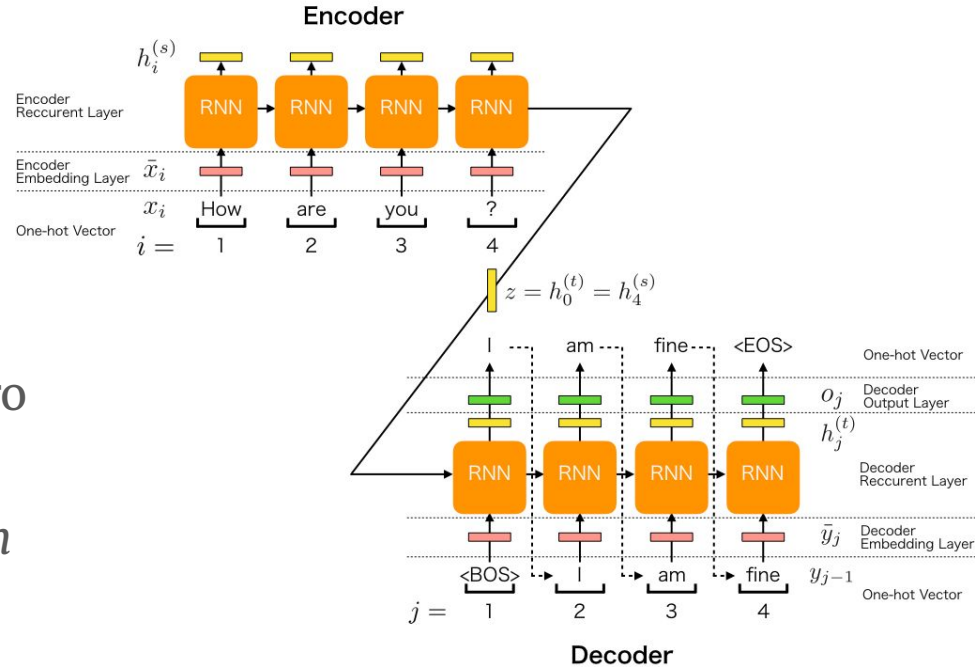
En el caso del entrenamiento se usa *teacher forcing* y en cada paso del decoder se ingresa la palabra correcta aunque no haya sido la predicha.



Seq2Seq Bottleneck

Esta arquitectura sufre de un problema de cuello de botella. Si pasamos un párrafo entero para traducir lo que se comparte entre el encoder y el decoder es un único vector h . En ese pequeño vector debemos codificar toda la información de la frase de entrada. Eso funciona bien para pequeñas frases pero no para muchas palabras.

En esta versión descartamos todos los h intermedios del encoder. ¿Por qué no dejamos que el decoder vea todos y decida cual utilizar?



Para solucionar este problema se introduce la noción de un *mecanismo de atención*.

Dejaremos que el decoder ponga su *atención* en la parte del input que le parezca razonable.

Mecanismos como el de atención tiene su origen varios años antes pero es en su versión de 2014 (Graves) que resulta utilizable en una NN.

Luego, en el trabajo de Bahdanau se utiliza por primera vez para evitar el problema del cuello de botella.

Neural Turing Machines

Alex Graves gravesa@google.com
Greg Wayne gregwayne@google.com
Ivo Danihelka danihelka@google.com

Google DeepMind, London, UK

Abstract

We extend the capabilities of neural networks by coupling them to external memory resources, which they can interact with by attentional processes. The combined system is analogous to a Turing Machine or Von Neumann architecture but is differentiable end-to-end, allowing it to be efficiently trained with gradient descent. Preliminary results demonstrate that *Neural Turing Machines* can infer simple algorithms such as copying, sorting, and associative recall from input and output examples.

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

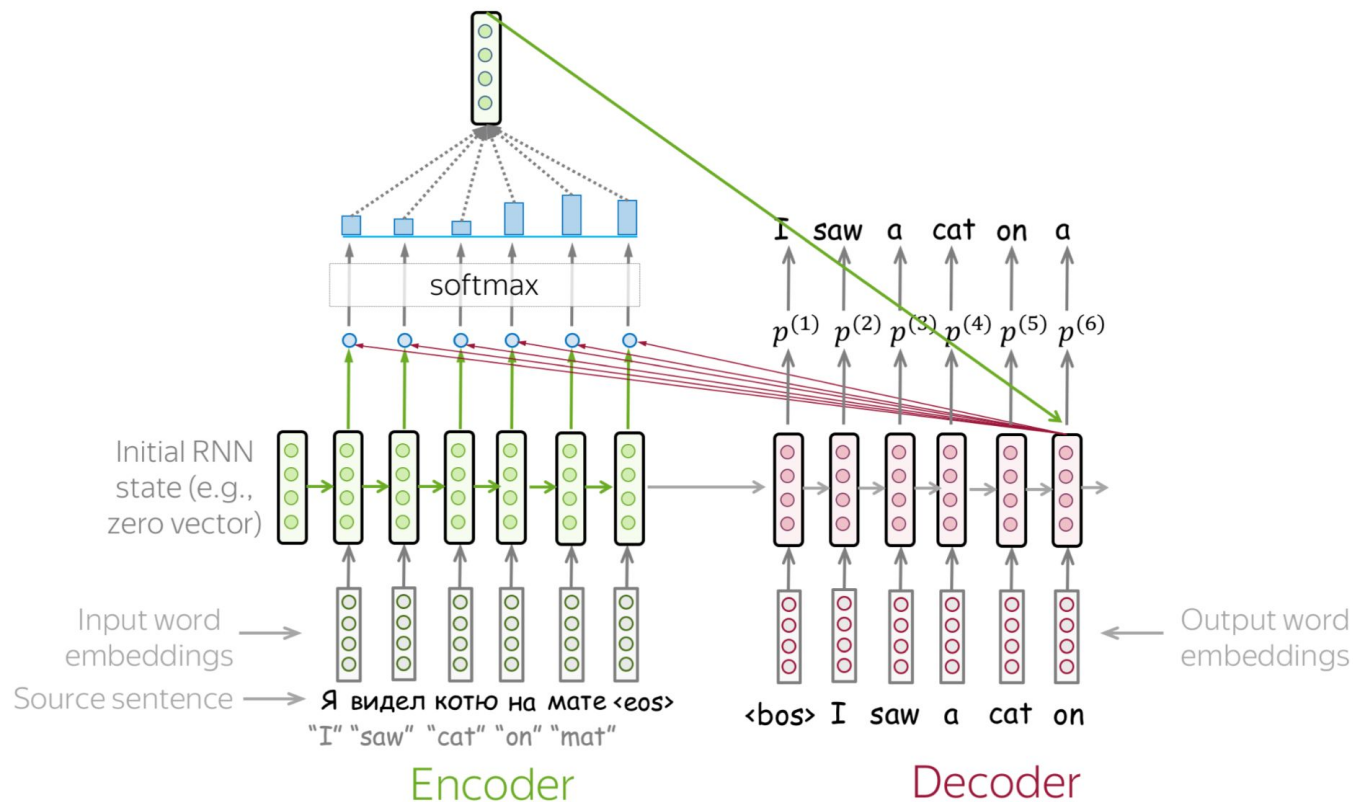
Dzmitry Bahdanau
Jacobs University Bremen, Germany

KyungHyun Cho Yoshua Bengio*
Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode a source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search for parts of a source sentence that are relevant to predicting a target word, without having to form these parts as a hard segment explicitly. With this new approach, we achieve a translation performance comparable to the existing state-of-the-art phrase-based system on the task of English-to-French translation. Furthermore, qualitative analysis reveals that the (soft-)alignments found by the model agree well with our intuition.

Attention



Attention

Supongamos que estamos en el punto de la traducción donde ya pasó toda la frase de entrada por el encoder y estamos generando la, por decir alguna, tercera palabra de la salida.

Como ya pasamos toda la frase de entrada por el encoder, ya tenemos todos los *hidden states* disponibles. Tenemos una matriz H con cada uno de los h_i con i yendo desde 1 hasta la longitud de la entrada (medida en palabras o tokens).

Además, como nuestro decoder es una RNN (particularmente una LSTM), tenemos su propio *hidden state* que llamaremos s_{t-1} , el estado que nos dejó la pasada anterior por el decoder.

Attention

Hay muchas maneras alternativas para computar *scores* de atención. Es decir, hay muchas maneras posibles para que una red aprenda dónde poner el foco en la entrada. Próximamente veremos la más utilizada actualmente, pero en el paper original usaron una capa lineal de la siguiente manera:

$$e_{t,i} = v_a^T \tanh(W_a s_{t-1} + U_a h_i)$$

De esta manera $e_{t,i}$ resulta en un escalar que representa el *score* de h_i en el paso t .

v_a , W_a y U_a son pesos aprendibles por la red.

Attention

Luego de calcular los $e_{t,i}$, se pasan los scores por una *softmax* para convertirlos en una distribución de probabilidad, exacerbando los scores calculados.

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^T \exp(e_{t,j})}$$

Attention

Por último, se utilizan los scores calculados para ponderar todos los vectores h del encoder.

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i$$

Si bien es una ponderación, al haber pasado por una *softmax* es posible que un vector sea claramente el ganador. La semántica es *“para dar la siguiente palabra de la traducción, enfocate en lo que sabíamos de la frase original en este punto en particular”*.

Una vez que calculamos este vector de contexto c_t hay diferentes maneras de utilizar en el siguiente paso del decoder. En el paper original esa manera era directamente concatenarlo con la entrada clásica a la red. Es decir, los vectores h y c de la LSTM se utilizaban de la misma manera pero la entrada en vez de ser solamente el embedding de la última palabra traducida, era eso más el vector de contexto c_t concatenado.

Seq2Seq con Attention

El mecanismo de Attention en la arquitectura encoder-decoder para problemas seq2seq fue un punto de quiebre importante en los benchmarks de Machine Translation.

En 2016 Google utiliza esta nueva arquitectura para entrenar su traductor oficial, haciendo el cambio definitivo de los traductores estadísticos a los NMT.

Alrededor de 2 o 3 años después todos los traductores importantes seguían el mismo camino.

Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi
yonghui,schuster,zhifengc,qvl,mnorouzi@google.com

Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, Jeffrey Dean

Abstract

Neural Machine Translation (NMT) is an end-to-end learning approach for automated translation, with the potential to overcome many of the weaknesses of conventional phrase-based translation systems. Unfortunately, NMT systems are known to be computationally expensive both in training and in translation inference – sometimes prohibitively so in the case of very large data sets and large models. Several authors have also charged that NMT systems lack robustness, particularly when input sentences contain rare words. These issues have hindered NMT's use in practical deployments and services, where both accuracy and speed are essential. In this work, we present GNMT, Google's Neural Machine Translation system, which attempts to address many of these issues. Our model consists of a deep LSTM network with 8 encoder and 8 decoder layers using residual connections as well as attention connections from the decoder network to the encoder. To improve parallelism and therefore decrease training time, our attention mechanism connects the bottom layer of the decoder to the top layer of the encoder. To accelerate the final translation speed, we employ low-precision arithmetic during inference computations. To improve handling of rare words, we divide words into a limited set of common sub-word units ("wordpieces") for both input and output. This method provides a good balance between the flexibility of "character"-delimited models and the efficiency of "word"-delimited models, naturally handles translation of rare words, and ultimately improves the overall accuracy of the system. Our beam search technique employs a length-normalization procedure and uses a coverage penalty, which encourages generation of an output sentence that is most likely to cover all the words in the source sentence. To directly optimize the translation BLEU scores, we consider refining the models by using reinforcement learning, but we found that the improvement in the BLEU scores did not reflect in the human evaluation. On the WMT'14 English-to-French and English-to-German benchmarks, GNMT achieves competitive results to state-of-the-art. Using a human side-by-side evaluation on a set of isolated simple sentences, it reduces translation errors by an average of 60% compared to Google's phrase-based production system.

Attention

Arrancamos con las RNN que no podían *recordar* ni 10 palabras y rápidamente llegamos a arquitecturas que mezclan Attention y LSTMs para poder traducir frases de una longitud muchísimo más extensa y con una gran precisión.

La duda final. Si el mecanismo de Attention nos deja aprender a poner el foco en cualquier parte del input (y eventualmente del output), ¿para qué necesitamos la arquitectura LSTM y sus vectores *hidden*?

Spoiler: No las necesitamos.